

Modern Web Development

The Svelte Framework

Welcome & Setup

Daniel Allmer

Senior Software Developer at axtesys.at

daniel.allmer@axtesys.at

Welcome & Setup

- Round table: name, background, what you hope to build
- Check Node.js: `node -v` (need v20+)
- Check IDE: VS Code + Svelte extension, Cursor, or Zed
- Course timeline & grading requirements
- Everything builds on the **same project** across all 3 days

Web Fundamentals & History

How the Web Works

- **Client / Server model:** browser sends HTTP request, server responds
- The browser is a document viewer that learned to run code
- **The three pillars:**
 - HTML — structure
 - CSS — presentation
 - JavaScript — behavior

HTML — Structure

```
<div class="card">  
  <h2>Welcome</h2>  
  <p>This is a simple card component.</p>  
  <button>Click me</button>  
</div>
```

CSS — Presentation

```
.card {  
  max-width: 320px;  
  padding: 1.5rem;  
  border-radius: 8px;  
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);  
  font-family: system-ui, sans-serif;  
}  
  
.card button {  
  background: #3b82f6;  
  color: white;  
  border: none;  
  padding: 0.5rem 1rem;  
  border-radius: 4px;  
  cursor: pointer;  
}
```

JavaScript — Behavior

```
<script>
  const button = document.querySelector('.card button');
  const heading = document.querySelector('.card h2');

  let count = 0;

  button.addEventListener('click', () => {
    count++;
    heading.textContent = `Clicked ${count} time${count === 1 ? '' : 's'}`;
  });
</script>
```

An HTTP Request

```
GET /about HTTP/1.1      ← method + path
Host: example.com        ← which server
Accept: text/html        ← what the browser wants
```

```
HTTP/1.1 200 OK          ← status code
Content-Type: text/html   ← what the server sends
```

```
<html>
  <body>Hello!</body>
</html>
```

- **Methods:** GET (read), POST (write), PUT , DELETE
- **Status codes:** 200 OK, 301 redirect, 404 not found, 500 server error

What the Browser Does

1. **Parses HTML** top-to-bottom → builds a DOM (Document Object Model)
2. **Fetches CSS & JS** in parallel as it encounters `<link>` and `<script>` tags
3. **CSS blocks rendering** — browser won't paint until stylesheets are processed
4. **Renders pixels** — layout, paint, display
5. **Runs JavaScript** — page becomes interactive

DevTools: See the Difference

Classic SSR — Wikipedia

- Open any Wikipedia article
- **View Source** — the entire article is right there in the HTML
- **Network tab** — initial document contains all content, minimal JS
- Pure server rendering — browser just displays what the server sent

CSR / SPA — Excalidraw

- Open <https://excalidraw.com>, open Network tab, hard reload
- **View Source** — almost empty: `<div id="root">` and script tags
- **Network tab** — tiny HTML, then dozens of JS bundles, then API calls
- The browser does all the work — server just handed it a blank page and JavaScript

Hybrid — svelte.dev/docs

- Open <https://svelte.dev/docs/svelte>
- **View Source** — full content is in the HTML (like Wikipedia!)
- Click a sidebar link, watch Network tab — no new document request, just a small JSON fetch
- **First visit:** server-rendered. **Navigation after:** client-side.

This is what SvelteKit gives you.

Rendering Strategies

Three Eras of Web Development

Era	Period	How it works	Examples
Server-rendered (MPAs)	~1995–2010	Server builds full HTML, sends to browser	PHP, Rails, Django
Client-rendered (SPAs)	~2010–2020	Server sends empty shell, JS renders everything	React, Vue, Angular
Hybrid (today)	~2020+	Server-render first page, client-side nav after	Next.js, Nuxt, SvelteKit

CSR & SSR

- **CSR (Client-Side Rendering)**

- Server sends empty HTML shell, JS renders in browser
- Pro: rich interactivity, no server after initial load
- Con: slow first paint, bad SEO, blank page until JS loads

- **SSR (Server-Side Rendering)**

- Server renders full HTML on each request
- Pro: fast first paint, good SEO, works without JS
- Con: server load on every request, slower page-to-page navigation

SSG, ISR & Hydration

- **SSG (Static Site Generation)**: pages pre-rendered at build time
 - Fastest possible, cheap hosting, great SEO
 - Content stale until next build, no per-user dynamic content
- **ISR (Incremental Static Regeneration)**: static pages that revalidate after a time interval
 - Static speed with fresher content
 - Added complexity, not all hosts support it
- **Hydration**: server sends rendered HTML, then JS "takes over" to make it interactive
 - This is what SvelteKit does by default: **SSR + hydration**

Hydration: How It Works

1. **Server** renders the component to HTML — user sees content immediately
2. HTML is sent to the browser along with a JS bundle
3. **Browser** loads JS, Svelte "hydrates" — attaches event listeners to existing DOM
4. Page is now **fully interactive**

The user never sees a blank page or a spinner.

The Modern Approach: Pick Per Route

Route	Strategy
Landing page	SSG
Dashboard	SSR
Interactive widget	CSR after hydration

SvelteKit lets you choose the right strategy **per route**.

Frameworks & Ecosystem

The Framework Landscape

Framework	Notes
React + Next.js	Biggest ecosystem, most jobs, RSC is the new paradigm
Vue + Nuxt	Similar to React/Next, gentler learning curve
Angular	Enterprise-focused, opinionated, batteries included
Svelte + SvelteKit	Compile-time, smallest bundles, closest to plain HTML/CSS/JS
Solid + SolidStart	Fine-grained reactivity, no virtual DOM, React-like syntax
Astro	Content-focused, "islands" architecture, great for blogs/docs
HTMX	HTML over the wire, minimal JS — niche but interesting

See also: [State of JS 2025 — Meta-Frameworks](#) | [Component Party](#)

Shared Tooling

- **TypeScript** — type safety
- **Vite** — build tool (used by Svelte, Vue, React, etc.)
- **npm / pnpm / bun** — package managers
- **Tailwind** — utility-first CSS

Key takeaway: learning one framework well makes it easy to pick up others.

Vite: The Build Tool

- **Dev server** — instant startup, hot module replacement (HMR)
- **Bundler** — optimizes code for production (tree-shaking, minification)
- Used by Svelte, Vue, React, Astro, Solid — it's the standard
- SvelteKit is built on top of Vite

```
npm run dev    → starts Vite dev server  
npm run build  → creates production bundle  
npm run preview → preview the production build locally
```

Final Project Brief & Ideas

The Project

- Each student builds their own app over the coming weeks (~14 hours of work)
- **Deadline: March 16, 2026**
- See [PROJECT.md](#) for full requirements and grading

Requirements:

- Landing page + additional pages
- Tailwind styling
- Supabase (auth for both lanes, database for Lane A)
- Deployed on Vercel

Choose Your Lane

Pick **one** lane — both are equally valid.

Lane A: Server-Side MPA

- Load functions, form actions, full CRUD
- Proper data management in Supabase
- Server-rendered pages

Lane B: Interactive Client

- Multiple `$state` values with `$derived` computations
- Rich client-side interactivity
- No database needed beyond auth

Three Questions for Tomorrow

1. What does the app do? (one sentence)
2. What are the different pages?
3. What data is managed by the app?

Lane A Ideas (Server-Side MPA)

Idea	CRUD operations
Recipe book	Create/edit/delete recipes, tag by category
Bookmark manager	Add/edit/delete bookmarks, manage tags
Event board	Create/edit/delete events, RSVP
Expense tracker	Add/edit/delete transactions, category management

Lane B Ideas (Interactive Client)

Idea	State & derived usage
Pomodoro timer	Time remaining, session count, break logic
Quiz app	Score, progress percentage, streaks
Workout planner	Total volume, duration, set counts
Budget calculator	Category totals, balance, savings rate

Pick something you'd actually use — motivation matters more than originality.

Buffer / Q&A

Break

SvelteKit Intro + Template Syntax

Create a New Project

```
npx sv create my-app  
cd my-app  
npm install  
npm run dev
```

Project Structure

```
my-app/
├── src/
│   ├── routes/
│   │   ├── +page.svelte    ← home page
│   │   ├── +layout.svelte ← shared layout
│   │   └── about/
│   │       └── +page.svelte ← /about page
│   ├── lib/                ← shared code ($lib alias)
│   └── app.html            ← HTML shell
├── static/                 ← public assets
├── svelte.config.js
├── vite.config.ts
└── package.json
```

File-Based Routing

The file system is the router — no config file needed.

```
src/routes/
├── +page.svelte          → /
├── about/
│   └── +page.svelte      → /about
├── dashboard/
│   └── +page.svelte      → /dashboard
├── posts/
│   ├── +page.svelte      → /posts
│   └── [id]/
│       └── +page.svelte  → /posts/1, /posts/2, ...
```

- Folder name = URL segment, `+page.svelte` = the page component
- **Dynamic routes:** `[brackets]` in the folder name become parameters
- **Layouts:** `+layout.svelte` wraps all pages at that level and below

Layouts

```
<!-- src/routes/+layout.svelte -->
<script>
  let { children } = $props();
</script>

<nav>
  <a href="/">Home</a>
  <a href="/about">About</a>
</nav>

{@render children}
```

- Layout wraps all pages at that level and below
- `{@render children}` is where the page content goes

Variables & Expressions

```
<script lang="ts">
  const name = 'World';
  const items = ['Apple', 'Banana', 'Cherry'];
</script>

<h1>Hello, {name}!</h1>
```

Just curly braces for expressions — no JSX, no special syntax.

Conditional Rendering: `{#if}`

```
<script lang="ts">  
  const loggedIn = false;  
</script>
```

```
{#if loggedIn}  
  <p>Welcome back!</p>  
{:else}  
  <p>Please log in.</p>  
{/if}
```

Loops: `{#each}`

```
<script lang="ts">  
  const items = ['Apple', 'Banana', 'Cherry'];  
</script>
```

```
<ul>  
  {#each items as item, i}  
    <li>{i + 1}. {item}</li>  
  {/each}  
</ul>
```

Exercise: Scaffold & Route

Exercise: Scaffold & Route

Goal: a running SvelteKit app with 3 pages, shared navigation, and use of `{#if}` and `{#each}`.

- Run `npx sv create` and pick a project name
- Create at least 3 routes (e.g., `/`, `/about`, `/dashboard`)
- Add a `+layout.svelte` with navigation links
- Use `{#each}` to render a list on at least one page
- Use `{#if}` to conditionally show/hide something
- Verify: clicking nav links changes the page **without a full reload**

Load Functions & SSR

The Problem

The page needs data (from a DB, API, etc.) **before** it can render.

Solution: `+page.server.ts` — runs on the server only, returns data to the page.

```
// src/routes/dashboard/+page.server.ts
import type { PageServerLoad } from '.$types';

export const load: PageServerLoad = async () => {
  const response = await fetch(
    'https://jsonplaceholder.typicode.com/posts?_limit=5'
  );
  const posts = await response.json();

  return { posts };
};
```

Accessing Data in the Page

```
<!-- src/routes/dashboard/+page.svelte -->
<script lang="ts">
  let { data } = $props();
</script>

<h1>Dashboard</h1>
{#each data.posts as post}
  <article>
    <h2>{post.title}</h2>
    <p>{post.body}</p>
  </article>
{/each}
```

Loading with Route Params

Remember `[id]` folders from routing? The load function receives the parameter via `params`.

```
// src/routes/posts/[id]/+page.server.ts
import type { PageServerLoad } from './$types';

export const load: PageServerLoad = async ({ params }) => {
  const response = await fetch(
    `https://jsonplaceholder.typicode.com/posts/${params.id}`
  );
  const post = await response.json();
  return { post };
};
```

Navigate to `/posts/1`, `/posts/2`, etc. — each gets different data.

Load Functions: Key Points

- `+page.server.ts` runs **only on the server** — safe for secrets, DB access
- Data is serialized as JSON and sent to the client
- Page is server-rendered with data already in place (fast first paint, good SEO)
- Also exists: `+page.ts` (universal load, runs on both) — we'll stick with server for now
- **Error handling:** use `error()` from `@sveltejs/kit` to throw HTTP errors, add `+error.svelte` to customize the error page

Exercise: Fetch & Display Data

Exercise: Fetch & Display Data

Goal: a page that fetches data from a public API and renders it with `{#each}`.

- Create a `+page.server.ts` for one of your routes
- Fetch from any public API:
 - `https://jsonplaceholder.typicode.com/posts`
 - `https://pokeapi.co/api/v2/pokemon?limit=20`
 - `https://dummyjson.com/products`
 - Or any API you find interesting
- Display data in a list using `{#each}`
- Use `{#if}` to handle the empty state
- **Bonus:** add a dynamic route with `[param]` for a detail page

Form Actions

Why Forms?

Progressive enhancement — works without JS, validated on the server.

```
// src/routes/guestbook/+page.server.ts
import type { Actions, PageServerLoad } from './$types';

const messages: { name: string; text: string }[] = [];

export const load: PageServerLoad = async () => {
  return { messages };
};

export const actions: Actions = {
  default: async ({ request }) => {
    const formData = await request.formData();
    const name = formData.get('name') as string;
    const text = formData.get('text') as string;
    messages.push({ name, text });
    return { success: true };
  }
};
```

The Form in the Page

```
<!-- src/routes/guestbook/+page.svelte -->
<script lang="ts">
  let { data, form } = $props();
</script>

<h1>Guestbook</h1>

<form method="POST">
  <input name="name" placeholder="Your name" required />
  <input name="text" placeholder="Your message" required />
  <button type="submit">Post</button>
</form>

{#if form?.success}
  <p>Message posted!</p>
{/if}

{#each data.messages as msg}
  <p><strong>{msg.name}</strong>: {msg.text}</p>
{/each}
```

Form Actions: Key Points

- No `fetch()` calls needed — native form submission
- The `form` prop contains the action's return value
- Works even if **JavaScript is disabled**
- Named actions (`action="?/delete"`) for multiple forms on one page
- Data is **in-memory** here — it resets when the server restarts. We'll use a real database on Wednesday.

Exercise: Add a Form

Exercise: Add a Form

Goal: a page with a working form that submits data via a form action.

- Pick a page in your project (or create a new route like `/guestbook`)
- Add a `+page.server.ts` with a `load` function and a `default` action
- Store submitted data in an in-memory array (we'll use a database on Wednesday)
- Display the submitted entries with `{#each}`
- Verify: submitting the form adds an entry to the list without manual refresh
- **Bonus:** add a second named action (e.g., `?/delete`) to remove entries

Wrap-Up

What We Covered Today

- How the web works & rendering strategies (CSR, SSR, SSG, hydration)
- The framework landscape
- SvelteKit project structure & file-based routing
- Template syntax: `{#if}`, `{#each}`, expressions
- Server-side data loading with `+page.server.ts`
- Form actions for mutations

For Tomorrow

- **Come with a project idea:**
 1. What does the app do? (one sentence)
 2. What are the different pages?
 3. What data is managed by the app?
- **Tomorrow's topics:** client-side state & reactivity, Tailwind CSS, project idea discussions